# Основные структуры данных (коллекции)

Лекция 3

Лектор: Половикова Ольга Николаевна

# Коллекции

Это встроенные типы данных, которые позволяют объединять по одним именем некоторую совокупность элементов (объектов).

В языке Python используются следующие встроенне коллекции:

**Список (list)**

**Множество (set)**

**Словарь (dict)**

**Кортеж (tuple)**

# Коллекции

Можно условно выделить две группы коллекций:  изменяемые, неизменяемые.

Если элементы коллекций может изменять то такие коллекции называются **изменяемыми** (изменяемый тип данных).

А коллекции, элементы которых нельзя изменить после создания коллекций, называется **неизменяемыми** (неизменяемый тип данных). Например,
**кортежи** – неизменяемые, а **словари** и **списки** изменяемые.

# Списки

Упорядоченные изменяемые коллекции объектов произвольных типов (почти как массив, но типы элементом могут отличаться).

```
# создаём список произвольных элементов
b = [1, 'information', 3.1457, 2**8]
# выводим список и его тип
print(b, type(b))
```

```
[1, 'information', 3.1457, 256] <class 'list'>
```

```
# Создание пустого списка -- два способа
d1 = list()
d2 = []
print(d1)
print(d2)
```

```
[]
[]
```

# Примеры задания Списков

Построение списка из строки с использованием конструктора *list()*

```
# можно создать список строк: разбив строку на подстроки
b = list('Список строк')
print(b)
```

```
['С', 'п', 'и', 'с', 'о', 'к', ' ', 'с', 'т', 'р', 'о', 'к']
```

Построение списка на основе копирования одного значения

```
# список из нулей
b = [0]*10
print('Вывод:', b)
```

```
Вывод: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

# Примеры задания Списков

## Список подстрок заданной строки

```
# из строки на основе разбиения по заданному символу
a = 'примеры решения задач'
b = a.split(' ')
print('Список:',b)

# обратная манипуляция: сцепление списка строк в одну строку
c = ':' # разделитель
c = c.join(b)
print('Строка:',c)
```

```
Список: ['примеры', 'решения', 'задач']
Строка: примеры:решения:задач
```

Функция *split* возвращает список строк, рабивая начальную строку по разделителю
Функция *join* возвращает строку, последовательно сцепляя все элементы списка (используя между ними разделитель), *string_name.join(разделитель)*

# Списковое включение (генератор списков)

Это продвинутая функция списков Python, делающая код более чистым и читабельным. Это способ выполнения ряда операций над списком с использованием одной строки. Включения обозначаются *выражением for в квадратных скобках*.

**Шаблон для спискового включения**:

```
новый_список =
[возвращаемое_значение for пременная in список логическое_выражение ]
```

**Примеры**:

Построение списка целых чисел от *a* до *d:*

```
[i for i in range(a, b)]
```

Построение списка целых чётных чисел от *a* до *d:*

```
[i for i in range(a, b) if i % 2 == 0]
```

# Примеры задания Списков

Построение списка целых случайных значений в диапазоне от *-10* до *10*

```
import random
# функция random.randint(0, i) возвращает случайное число в заданном диапазоне
b = [random.randint(-10, 10) for i in range(10)]
print(b)
```

```
[0, -4, 1, -8, -4, -8, 1, 8, -2, 6]
```

Иструкция *import random* подключает библиотеку для работы со случайными числами

Список случайных вещественных величин

```
# подключаем библиотеку
import random
# задаём границы вещественного интервала
x, y = float(input()), float(input())

# генерируем вещественные случайные числа и складываем их в список
b = [round(random.uniform(x, y), 2) for i in(range(10))]
print(b)
```

```
-5.5
12.8
[9.4, -2.45, -0.83, 2.38, 11.96, -2.53, -3.77, 6.07, 2.07, 5.12]
```

**Ввод значений для x и y**

Функция round(number[, ndigits]) - округляет число *number* до *ndigits* знаков после запятой (по умолчанию, до нуля знаков, то есть, до ближайшего целого)

Функция *random.uniform()* используется для генерации числа с плавающей запятой в пределах заданного промежутка

# Доступ к элементу Списка

Элементы списка пронумерованы (проиндексированы) начиная с *0* индекса:

```
# Обращение к элементу списка по его индексу
b = [1, 'information', 3.1457, 2**8, ['a','b']]
print(b[0])
print(b[0] + b[2] + b[3])
print(b[4][1])
```

```
1
260.1457
b
```

Python также поддерживает отрицательную индексацию. Отрицательная индексация начинается с последнего элемента. Последний элемент списка имеет индекс *-1*. Используя отрицательную индексацию можно получить доступ к последним элементам не используя дополнительные функции.

| **b =** | **1** | **‘information’** | **3,1457** | **64** | **[‘a’, ‘b’]** |
|---|---|---|---|---|---|
| индекс | 0 | 1 | 2 | 3 | 4 |
| индекс | -5 | -4 | -3 | -2 | -1 |

```
# отрицательные индексы элементов списка
b = [1, 'information', 3.1457, 2**8, ['a','b']]
print(b[-1], b[-2])
```

```
['a', 'b'] 256
```

# Изменение элемента Списка

Список – это изменяемый тип данных, поэтому при обращении по индексу к его элементу можно заменить его знчение (и соответственно тип)

```
# изменение значения элемента Списка
b = [1, 'information', 3.1457, 2**8, ['a','b']]
print(b)
b[-1] = 8 ** 2
print(b)
```

```
[1, 'information', 3.1457, 256, ['a', 'b']]
[1, 'information', 3.1457, 256, 64]
```

## Реверс или зеркалирование элементов списка

```
# Изменим порядок элементов в списке: зеркалирование
b = list('коту тащат')
print(b)
# функция len - возвращает кол-во элементов в списке (длину)
for i in (range(len(b) // 2)):
    temp = b[i]
    b[i] = b[-i-1]
    b[-i-1] = temp
print(b)
print(''.join(b))
```

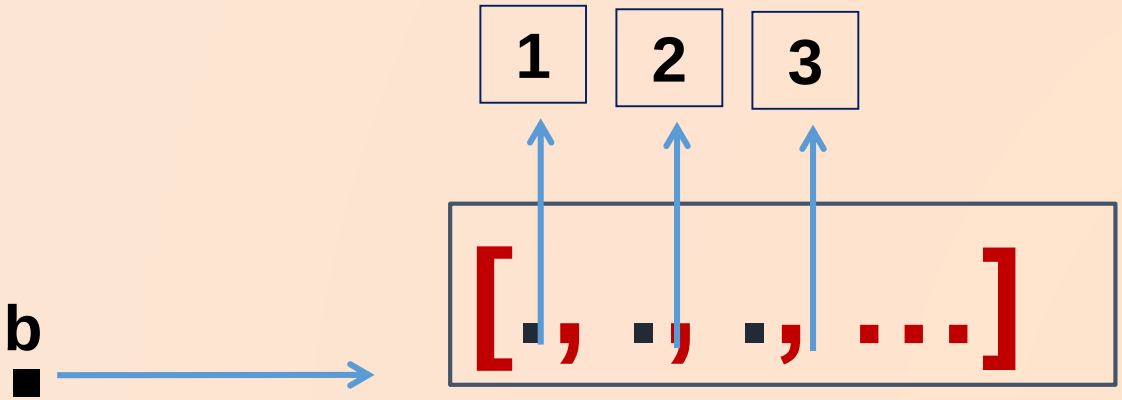
```
['к', 'о', 'т', 'у', ' ', 'т', 'а', 'щ', 'а', 'т']
['т', 'а', 'щ', 'а', 'т', ' ', 'у', 'т', 'о', 'к']
тащат уток
```

# Хранение Списков

При создании Списка в памяти резервируется область, которую можно условно назвать некоторым «контейнером», в котором хранятся ссылки на другие элементы данных в памяти (на элементы списка).

**Содержимое данного «контейнера» можно менять** – поэтому Списку относятся к изменяемыму типу.

Идентификатор **имени Списка** – это **ссылка** на резервируемую область памяти, на «контейнер».



# Копирование Списков

Для копирования списков можно использовать  слудующие функции и выражения:

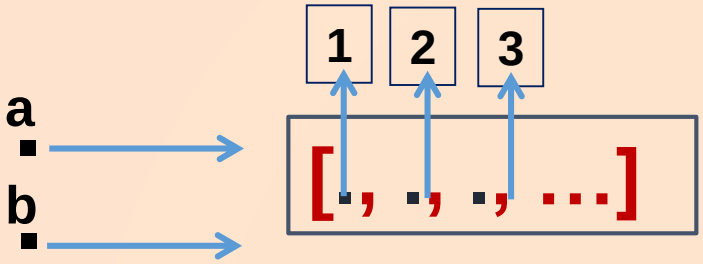
```
# заполняем список целочисленными значениями от 0 до 9 ( < 10)
b = list(range(10))
print(b)
# копируем список
a = list(b) # или a = b[:] или a = b.copy()
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

В случае выполнения простого присваивания списков друг другу, то переменной *a* будет присвоена ссылка на тот же элемент данных в памяти, на который ссылается *b*, а не копия списка *b*. Т.е. если вы будете изменять список *a*, то и *b* тоже будет меняться (и наоборот).

```
# заполняем список целочисленными значениями от 0 до 9 ( < 10)
b = list(range(10))
a = b
a[0] = a[-1]
print(a)
print(b)
```

```
[9, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[9, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



# Срезы

**Срезы** позволяют выбирать из списка блок последовательных элементов, который в данный момент нужен. Срезы работают по схеме:

список[начало: конец: шаг], где

Начало - с какого элемента выбираем блок(по умолчанию равна 0);

Конец - до какого элемента выбираем блок (по умолчанию равно длине списка)

Шаг - с каким шагом берем элементы (напимер, каждый *2* или *3* (по умолчанию каждый *1*).

**список[:: 2]** – берём каждый второй элемент

**список[-1:: -2]** – начиная с последнего элемента берем каждый второй элемент

**список[2:4: ]** – начиная со 2-ого элемента берем все элементы до 4-ого элемента (два элемента)

```
# создаём список конструктором, перебирвая все целые величитны от 0 до 10
b = list(range(10))
print('b=',b)
# делаем срезы
print('b[::2]= ',b[::2])
print('b[-1::-2]= ',b[-1::-2])
print('b[2:4]= ',b[2:4])
```

```
b= [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
b[::2]=  [0, 2, 4, 6, 8]
b[-1::-2]=  [9, 7, 5, 3, 1]
b[2:4]=  [2, 3]
```

# Перебор элементов Списка

Перебирать элементы списка можно с помощью цикла, например цикла *for:*

*for имя_переменной in список*

Решим задачу. Для некоторого набора целых положительных чисел необходимо определить количество четных и нечётных значений.
Замечание. *Набор положительных чисел зададим через консольный ввод*.

```
# зададим список целых чисел
b = [int(i) for i in input().split(' ')]
print(b)
even, odd = 0, 0 # счётчики чётных и нечётных величин
for i in b:
    if (i > 0) and (i % 2 == 1):
        even += 1
    if (i > 0) and (i % 2 == 0):
        odd +=1
print(even, odd)
```

```
1 2 3 12 0 -2 22
[1, 2, 3, 12, 0, -2, 22]
2 3
```

ВВОД

Выражение *b = [int(i) for i in input().split(' ')]*, позволяет сформировать список целых чисел *b*. Функция input() – возращет строковое представление вводимых данных. Выражение *input().split(' ')* – возращет подстроки данной строки по разделителю «пробел». Цикл *for i in input().split(' ')* – выполняет перебор по подстрокам. Метод *int(i)* – преобразет подстроку (строковое представление числа) в целое число.

# Действия над списками

| | |
|---|---|
| x in список | True, если элемент x находится в списке |
| x not in список | True, если элемент x не находится в списке |
| список1 + список2 | Объединение списков |
| список * n или<br>n * список | Копирует список n раз |
| len(список) | Длина списка, т.е. количество элементов в списке |
| min(список) | Наименьший элемент списка |
| max(список) | Наибольший элемент списка |
| sum(список) | Сумма чисел списка |
| sorted(список) | Сортировка элементов списка |

# Пример

Решем предыдущую задачу другим способом, используя списковое включения и функцию len():

```
# зададим список целых чисел
b = [int(i) for i in input('ввод чисел через пробел:').split(' ')]
print(b)
# количество чётных
even = len([i for i in b if (i > 0) and (i % 2 == 1)])
# количество нечётных
odd = len([i for i in b if (i > 0) and (i % 2 == 0)])
print(even, odd)
```

```
ввод чисел через пробел:2 4 5 8 13 5 0 -7 89 90 -12
[2, 4, 5, 8, 13, 5, 0, -7, 89, 90, -12]
4 4
```

# Пример

Задача. Пусть дан некоторый набор целочисленных значений, определить нименьшее значение среди чисел, которые строго больше среднего арифметического значения.

```
# ввод числовых значений, построение списка конструктором
b = list( int(i) for i in input('ввод чисел через пробел:').split(' '))
middle = sum(b)/len(b)
print(middle) # покажим среднее значение
minb = max(b) # инициализация
for i in b:
    if i > middle and i < minb:
        minb = i
print(minb)
```

```
ввод чисел через пробел:5 6 17 5 9 -9 12 8 9 14 0 11
7.25
8
```

Находим среднее значения, используя функции sum() и len(). Затем в цикле находим наименьшее значение, которое больше среднего арифметического значения

# Пример

Решим предыдущую задачу, используя списковое включение:

```
# ввод числовых значений, построение списка конструктором
b = list( int(i) for i in input('ввод чисел через пробел:').split(' '))
middle = sum(b)/len(b)
print(middle) # покажим среднее значение
minb = min([i for i in b if i > middle])
print(minb)
```

```
ввод чисел через пробел:5 6 17 5 9 -9 12 8 9 14 0 11
7.25
8
```

# Методы Списков

| Метод | Назначение |
|---|---|
| append() | Добавляет элемент(ы) в конец списка |
| clear() | Удаляет все элементы в списке |
| copy() | Возвращает копию списка |
| count() | Возвращает число элементов с определенным значением |
| extend() | Добавляет элементы в конец текущего списка |
| index() | Возвращает индекс первого элемента с определенным значением |
| insert() | Добавляет элемент по индексу |
| pop() | Удаляет элемент по индексу или последний |
| remove() | Удаляет элемент по значению |
| reverse() | Разворачивает список |
| sort() | Сортирует список |

# Использование методов

Синтаксическая конструкци вызова метода:
название_списка.название_метода(…)
Например,
*b = []*
*b.append(-1) #* добавляет новый элемен в конец списка *b*

Рассмотрим пример удаления элемента списк по значению

```
# Удаление из списка первого элемента с данным значением.
# будем работать со списком строк

b = list(input('Элементы через пробел : ').split(' ')) # список подстрок
x = input('элемент:') # искомый элемент
if x in b:
    b.remove(x)
print(b)
```

```
Элементы через пробел : 1 2 3 4 5 1
элемент:1
['2', '3', '4', '5', '1']
```

Метод *remove(значение)* удаляет первый элемент с указанным значением

# Сортировка Списка

Есть встроенная функция *sorted(список)* для сортировки итерируемых объектов в том числе и списков и метод *список.sort()* для сортировки списка с заменой исходного.

**# sorted(список, [key=ключ сортировки],[reverse=False])**

**# список.sort([key=ключ сортировки],[reverse=False])**

У функций: *список.sort()* и *sorted()* есть параметр *reverse*, принимающий *boolean*-значение. Он нужен для обозначения сортировки по убыванию.

```
# Сортировка списка список.sort([key=ключ сортировки],[reverse=False])

b = [-11, 3 , 0, 8, 20, -9, -15]
b.sort()
print('Прямой порядок: ',b)
b.sort(reverse=True)
print('Обратный порядок: ',b)
# зеркалирование элементов:
b.reverse()
print('Прямой порядок: ',b)
```

```
Прямой порядок:  [-15, -11, -9, 0, 3, 8, 20]
Обратный порядок:  [20, 8, 3, 0, -9, -11, -15]
Прямой порядок:  [-15, -11, -9, 0, 3, 8, 20]
```

# Сортировка Списка

Сортировать можно не только числовые списки, но и списки строк (лексикографически), списки списков:

```
# Сортировка списка sorted([key=ключ сортировки],[reverse=False])

b = 'Иванов Петров Сидоров Иванчик Ивонин'.split(' ')
print('Прямой порядок: ', sorted(b))
# список b не изменился
print('Первоначальный список: ',b)
```

```
Прямой порядок:  ['Иванов', 'Иванчик', 'Ивонин', 'Петров', 'Сидоров']
Первоначальный список:  ['Иванов', 'Петров', 'Сидоров', 'Иванчик', 'Ивонин']
```

# Сортировка Списка

**Если сортируется список списков** (в нашем случе, элемент списка – это список), то для сравнения таких сложных элементов сравниваются его составные части по правилу:

- если первые части различны, то результат определяется по первым частям,
- если первые составные чисти равны, то сраниваются вторые и т.д.

```
# Сортировка списка списков (элемент списка тоже список)
b = [[8, 2, 3], [1, 30, 50], [1, 30, 40]]
print('Прямой порядок: ', sorted(b))
# список b не изменился
print('Первоначальный список: ',b)
```

```
Прямой порядок:  [[1, 30, 40], [1, 30, 50], [8, 2, 3]]
Первоначальный список:  [[8, 2, 3], [1, 30, 50], [1, 30, 40]]
```

# Сортировка Списка

Для сравнения элементов можно использовать lambda-выражения в качестве ключа:

key =
lambda имя_для_элемента_списка : выражение с именем_для_элемента_списка,

**Имя для элемента списка – любое имя меременной**

```
# Сортировка списка с использованием ключа
b = [-11, 3 , 0, 8, 20, -9, -15]
# сортировка числовых величин по модулю
newb = sorted(b, key = lambda num: abs(num))
print('Прямой порядок: ', newb)
# список b не изменился
print('Первоначальный список: ',b)
```

```
Прямой порядок:  [0, 3, 8, -9, -11, -15, 20]
Первоначальный список:  [-11, 3, 0, 8, 20, -9, -15]
```

В данном примере происходит сортировка элементов списка по модулю (по абсолютному значению). В качестве имени для элемента списка используется идентификатор: ***num***, выражение: ***abs***(***num***)

# Кортеж (tuple)

Кортеж – это **последовательность неизменяемых объектов**. Кортежи очень схожи со списками, с той разницей — что элементы кортежа не могут быть изменены (так же используют обычные скобки, вместо квадратных, как в списках).

Кортежи следует использовать для случая, когда **совокупность объектов**, с которой вынуждена работать программа **не должна изменяться**. Тем самым, использование кортежей выступает **гарантом безопасности** двнных от изменения, потому что любая попытка изменить элемен кортежа, приводит к ошибке программы.

Кортежи более экономичны по ресурсам по сранению со списками. Кортеж в памяти занимают меньший объем по сравнению со спискам (с аналогичными значениями). Перебор элементов кортежа выполняется быстрее, чем у аналогичного списка.

**Создание кортежа:**

```
t = (1, 2, 3) # однотипных элементов
t = (1, 2.4, 'list', [1, 2]) # разных типов
t = () или t = tuple() # пустой кортеж
b = [1, 2, 3]
t  = (b) или t = tuple(b) #  на основе списка
```

# Кортеж (tuple)

Создадим кортеж перебором элемнтов от 0 до 10, сравним с аналогичным списком:

```
# список
b = [i for i in range(10)]
print(b)
# кортеж на основе конструктора tuple(перебор элементов)
tuple_b = tuple([i for i in range(10)])
print(tuple_b)
# типы
print(type(b), type(tuple_b))
# объём памяти
print('объём памяти ', b.__sizeof__(), tuple_b.__sizeof__())
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
<class 'list'> <class 'tuple'>
объём памяти  84 52
```

Объём памяти существенно отличается

**Большинство манипуляций со списками аналогично работают с кортежами: методы, срезы и т.д.**

# Работа с кортежем

Перебор элементов

Задача. Опеделить количество чётных элементов в некоторой числовой последовательности, заданной случайным образом

```
import random
# кортеж случайных элементов на основе конструктора tuple()
tuple_b = tuple([random.randint(-100, 100) for i in range(10)])
print(tuple_b)
counter = 0 #счётчик чётных элементов
# кол-во чётных элементов
for x in tuple_b:
    if x % 2 == 0:
        counter += 1
print(counter)
```

```
(55, 62, -71, 66, 34, 38, 4, -53, -5, -61)
5
```

Доступ к элементу по индексу.

```
t = (1, 2, 3)
print(t[-1])
```

**Изменять элементы нельзя**

# Множество (Set)

Тип **Множестово** – это неупорядоченная коллекция из нуля или более ссылок на объекты. **Множество** – контейнер, содержащий **неповторяющиеся элементы в случайном порядке**.

**Создание множества**:

# Создание пустого множества
s = set()

# Создание множества из списка
s = set([1, 2, 3, 10])

# Создание множества из строки
s = set('qwerty')

```
# Создание пустого множества
s = set()
print(s)

# Создание множества из списка

s = set([1, 2, 3, 4, 10])
print(s)

# Создание множества из строки
s = set('qwerty')
print(s)
```

```
set()
{1, 2, 3, 4, 10}
{'y', 't', 'w', 'r', 'e', 'q'}
```

# Множество (Set)

С множествами можно выполнять множество операций: принадлежность элемента множеству, находить мощность и т.д.

```
# Проверка наличия элемента во множестве
s = set([1, 2, 3, 4, 10])
print(s)
print('4 in s = ', 4 in s)
print('5 in s = ', 5 in s)

# Мощность множества
print('len(s) = ', len(s))

# Пересечение множеств
s1 = set([5, 6, 4, 2, 1])
print('\n s & s1: ', s & s1)
```

```
{1, 2, 3, 4, 10}
4 in s =  True
5 in s =  False
len(s) =  5

 s & s1:  {1, 2, 4}
```

Используя множество, можно избавляться от повторяющихся элементов (от дублей):

```
b = [7, 8, 7, 6, 10, 11, 10, 1, 7]
s = set(b)
print('Избавились от дублей:',s)
# Вывести количество различных элементов
print('количество различных элементов: ', len(set([1, 2, 3, 4, 1, 6, 3, 2])))
```

```
Избавились от дублей: {1, 6, 7, 8, 10, 11}
количество различных элементов:  5
```

# Множество (Set)

Множества можно объединять,  пересекать, находить разность, …

```
# Операции на множествах
a = set([10, 9, 8, 5, 6])
b = set([5, 6, 7, 11])
print('Объединение', a | b)  # Объединение
print('Пересечение', a & b)  # Пересечение
print('Разность: a без b', a - b)  # Разность: a без b
print('Симметричная разность', a ^ b)  # Симметричная разность
```

```
Объединение {5, 6, 7, 8, 9, 10, 11}
Пересечение {5, 6}
Разность: a без b {8, 9, 10}
Симметричная разность {7, 8, 9, 10, 11}
```

# Операции с множествами

| Операция/Метод | Назначение |
|---|---|
| len(s) | число элементов в множестве (размер множества) |
| x in s | принадлежит ли x множеству s |
| set.isdisjoint(other) | истина, если set и other не имеют общих элементов. |
| set == other | все элементы set принадлежат other, все элементы other принадлежат set |
| set.issubset(other) или<br>set <= other | все элементы set принадлежат other |
| set.union(other, ...) или<br>set \| other \| ... | объединение нескольких множеств |
| set.intersection(other, ...) или<br>set & other & ... | пересечение |
| set.difference(other, ...) или<br>set - other - ... | множество из всех элементов set, не принадлежащие ни одному из other |
| set.symmetric_difference(other);<br>set ^ other | множество из элементов, встречающихся в одном множестве, но не встречающиеся в обоих |
| set.copy() | копия множества |

# Операции, которые изменяют текущее множество

| Операция/Метод | Назначение |
|---|---|
| set.update(other, ...)<br>set \|= other \| ... | объединение |
| set.intersection_update(other, ...)<br>set &= other & ... | пересечение |
| set.difference_update(other, ...)<br>set -= other \| ... | вычитание |
| set.add(elem) | добавляет элемент в множество |
| set.remove(elem) | удаляет элемент из множества. Ошибка, если такого элемента не существует |
| set.discard(elem) | удаляет элемент, если он находится в множестве |
| set.pop() | удаляет первый элемент из множества.<br>Так как множества не упорядочены, нельзя точно сказать, какой элемент будет первым |
| set.clear() | очистка множества |

# Словарь (Dict)

**Словарь** – это неупорядоченная коллекция из нуля или более пар «**ключ-значение**», в которых в качестве ключей могут использоваться ссылки на неизменяемые объекты, а в качестве значений – ссылки на объекты любого типа.
Словарь относятся изменяемым типам данных, можно добавлять и удалять его элементы (**ключ-значение**).

```
# Создание пустого словаря
# Так создаются именно словари, а не множества
d1 = {}
print(d1, type(d1))
# ещё способ
d2 = dict()
print(d2, type(d2))
```

```
{} <class 'dict'>
{} <class 'dict'>
```

# Создание словарей:

Создать словарь можно с помощью литерала (используя фигурные скобки), конструктором: dict(…):

```
# Примеры создания словарей
d0 = dict(year=2002, newyeare=2020)
d1 = {"year": 2000, "city": "Москва", "lastname": "Иванов"}
d2 = dict(year = 2000, city = "Москва", lastname = "Иванов")
d3 = dict([("year",2000), ("city", "Москва"), ("lastname", "Иванов")])

print(d0)
print(d1)
print(d2)
print(d3)
```

```
{'year': 2002, 'newyeare': 2020}
{'year': 2000, 'city': 'Москва', 'lastname': 'Иванов'}
{'year': 2000, 'city': 'Москва', 'lastname': 'Иванов'}
{'year': 2000, 'city': 'Москва', 'lastname': 'Иванов'}
```

# Создание словарей:

Создать словарь можно генератором словарей (аналогично списковым включениям):

```
str = 'hhguuiersssssssqw'
# строка.count(символ) -- возвращает количество данного символа в строке
d1 = {char: str.count(char) for char in str}
d2 = {i: i ** 2 for i in range(8)}
print(d1)
print(d2)
```

```
{'h': 2, 'g': 1, 'u': 2, 'i': 1, 'e': 1, 'r': 1, 's': 7, 'q': 1, 'w': 1}
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49}
```

# Изменение словарей

Добавление элементов (ключ-значение) в словарь:

```
# Добавление в словарь еще одного значения или изменение значения  на новое
d1 = dict(year = 2000, city = "Москва", lastname = "Иванов")
print(d1)
d1["newyear"] = 2001
d1["year"] = 2002
print(d1)
```

```
{'year': 2000, 'city': 'Москва', 'lastname': 'Иванов'}
{'year': 2002, 'city': 'Москва', 'lastname': 'Иванов', 'newyear': 2001}
```

При добавление в словарь элементов, у которых ключи совпадают с ключами элементов из соваря, происходит изменение значений элементов, записанных ранее.

# Изменение словарей

Удаление элемента возможно по ключу:

```
# Удаление элемента из словаря
d1 = dict(year = 2000, city = "Москва", lastname = "Иванов")
del d1["year"]
print(d1)
d1.pop("city")
print(d1)
```

```
{'city': 'Москва', 'lastname': 'Иванов'}
{'lastname': 'Иванов'}
```

Изменить ключ элемента нельзя, но можно удалить данный элемент и добавить новый (с новым ключом).

```
# Проверка на наличие ключа в словаре
d1 = dict(year = 2000, city = "Москва", lastname = "Иванов")
print("year" in d1)
print("newyear" in d2)
```

```
True
False
```

# Методы словарей

| Метод | Назначение |
|---|---|
| dict.clear() | очищает словарь |
| dict.copy() | возвращает копию словаря |
| dict.get(key[, default]) | возвращает значение ключа, но если его нет, не бросает исключение, а возвращает default |
| dict.items() | возвращает пары (ключ, значение) |
| dict.keys() | возвращает ключи в словаре |
| dict.pop(key[, default]) | удаляет ключ и возвращает значение. Если ключа нет, возвращает default (по умолчанию бросает исключение) |
| dict.popitem() | удаляет и возвращает пару (ключ, значение). Если словарь пуст, бросает исключение KeyError (словари неупорядочены) |
| dict.setdefault(key[, default]) | возвращает значение ключа, но если его нет, не бросает исключение, а создает ключ с значением default |
| dict.values() | возвращает значения в словаре |

# Пример

Вывод всех элементов словаря. Создание списка на основе словаря

```
# Вывод словаря
d1 = dict(year = 2000, city = "Москва", lastname = "Иванов")
for x in d1.items():
    print(x[0], x[1]) #ключ и значение
# формирование списка списков на основе словаря
d = list()
for x in d1.items():
    d.append([x[0], x[1]])
print("\nсписок списков:", d)
```

```
year 2000
city Москва
lastname Иванов

список списков: [['year', 2000], ['city', 'Москва'], ['lastname', 'Иванов']]
```